

Lab 3

Component Modelling & Architectural Pattern Selection

Problem Statement #31

Multi-Vendor Artisan E-Commerce Marketplace

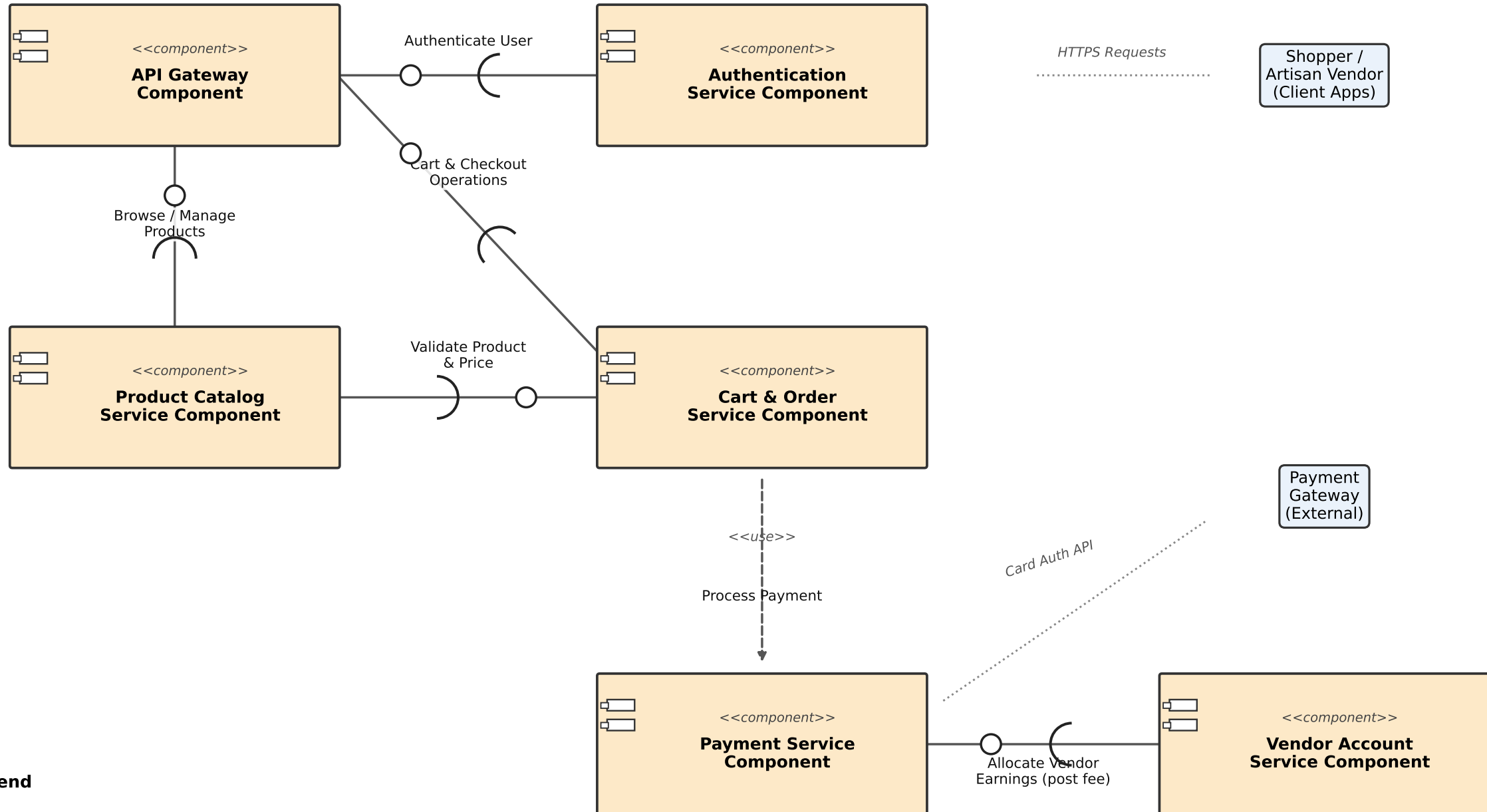
Name: Darshil Kachhadiya

SRN: PES1UG25CS820

GitHub: <https://github.com/Darshil085/SE--Labs--PES1UG25CS820-->

Multi-Vendor Artisan E-Commerce Marketplace

UML Component Diagram — Microservices Architecture



Legend

- Provided Interface (ball)
- ⌋ Required Interface (socket)
- <<use>> dependency
- External system interaction

Lab 3: Component Modelling & Architectural Pattern Selection

Written Architectural Justification

Problem Statement #31: Multi-Vendor Artisan E-Commerce Marketplace

Name: Darshil Kachhadiya **SRN:** PES1UG25CS820

GitHub: <https://github.com/Darshil085/SE--Labs--PES1UG25CS820-->

Architecture Selection

We chose **Microservices Architecture** for the Multi-Vendor Artisan E-Commerce Marketplace System.

Reason 1 — Independent Vendor-Facing Scaling

The system must support many Artisan Vendors independently managing their own product catalogs (FR002) while Shoppers simultaneously browse, search and add items from multiple vendors to a single cart (FR003, FR004). Splitting these responsibilities into separate services (Product Catalog Service, Cart & Order Service) allows the catalog service — which sees far higher read traffic and must meet the sub-500 ms CDN-cached image load requirement (NFR001) — to be scaled and deployed independently of order and checkout logic, without redeploying or overloading the rest of the system during traffic spikes such as festive sales.

Reason 2 — Fault Isolation for Critical Payment Flows

Order payments must be split across multiple independent vendor accounts after a 5% platform-fee deduction (FR001), which is the most business-critical and failure-sensitive operation in the system. By isolating this logic inside a dedicated Payment Service Component and Vendor Account Service Component, a fault or slowdown in the Product Catalog or Authentication services does not affect an in-progress payment, and a failure in payment processing does not take down product browsing — directly supporting the alternate "Handle Payment Failure" flow identified in the Lab 1 use-case model.

Security Advantage

Authentication and authorization are consolidated into a single, independently deployable Authentication Service Component that is the sole required interface every other service (via the API Gateway) must satisfy before accessing protected shopper or vendor account functions (NFR002). Centralizing identity verification in one microservice — rather than duplicating auth logic across the catalog, cart and payment layers — shrinks the attack surface, makes it easier to enforce a single consistent security policy (token validation, session expiry, role checks), and means a security patch only needs to be applied and audited in one place.

Performance Benefit

Because each microservice owns its own scaling profile, the Product Catalog Service can be replicated behind a CDN purely to satisfy the <500 ms image-load NFR (NFR001), while the Payment Service can be scaled and resourced separately to handle the heavier, synchronous computation of splitting cart totals across multiple vendor payouts (FR001). Neither workload competes for the same compute pool, so a surge in product browsing during a sale does not slow down checkout and payment confirmation for shoppers already at the cart stage.

Supporting artifact: See the accompanying UML Component Diagram (component_diagram.pdf) showing the 6 identified components — API Gateway, Authentication Service, Product Catalog Service, Cart & Order Service, Payment Service and Vendor Account Service — along with the 6 required/provided interfaces and the <<use>> dependency between Cart & Order Service and Payment Service.